A Fault Modeling Based Runtime Diagnostic Mechanism for Vehicular Distributed Control Systems

Alexandre Dos Santos Roque, *Member, IEEE*, Nasser Jazdi, *Member, IEEE*, Edison Pignaton De Freitas, *Member, IEEE*, and Carlos Eduardo Pereira, *Member, IEEE*

***Abstract*— This paper presents a study about a runtime mechanism to monitor the performance degradation in intra-vehicular networks. The proposed mechanism focuses on the integration of fault modeling in communication protocols, as non-functional requirements (NFR), using aspect-oriented modeling (AOM), to model the performance degradation generated by faults, linking test and design phases of distributed control systems. A case study analyzing the mechanism performance in both CAN and CAN-FD protocols was conducted considering the NFR specification related to fault disturbances. In order to evaluate and simulate the mechanism under real fault scenarios, an active suspension control system was considered as an example of a critical control system and faults were injected using the hardware Vector VH6501 (CAN disturbance interface). The network performance analysis was made based on the software Vector CANoe considering different network busloads and CAN/CAN-FD rates between 1 to 4 Mbps. Results show that the mechanism efficiently detects anomalous events on performance with short response time with busloads up to 30%. The performed experiments show better performance on CAN-FD registering lower average jitter during fault injection in all busloads tested scenarios.**

***Index Terms*— Performance diagnostic mechanisms, fault modeling, communication protocols, vehicular control systems.**

I. Introduction

DISTRIBUTED vehicular control systems are increasing in complexity leading to new demands related to fault detection and diagnosis, particularly due to safety-critical applications. In intra-vehicular networks, the communication protocols have an essential role of interconnecting ECUs (electronic control units) responsible to integrate different critical control systems (i.e. Active Suspension, Traction Control, Obstacle Avoidance, among other systems) [1], [2].

Protocols as FlexRay, CAN, LIN and MOST [3], [4] are commonly applied to interconnect ECUs, in which some of them are responsible for controlling critical processes [5]. However, with the recent technologies used to add intelligence and safety to automotive systems, there has been an increase in the message traffic, consequently increasing the possibilities of faults and interferences, also considering the occurrence of malicious attacks [6], [7].

Despite these constant improvements, the complexity inherent in today's distributed and real-time embedded systems also requires the development of solutions that could enable these protocols to detect and diagnose disturbances in performance of critical control systems, avoiding critical situations [8], [9]. Detecting and diagnosing different fault types is not a simple task and usually requires mechanisms implemented in hardware and software, besides having to understand the situations and the scenarios in which the performance degradation occurred due to certain faults.

In this sense, the CAN-FD (CAN with Flexible Data Rate, Robert Bosh - GmbH) [10], is a prominent protocol because it is able to address the demand for higher communication bandwidth and the need to meet the requirements of modern vehicular control systems. The main CAN-FD features include the increasing speed (1 to 8 Mbps), the amount of transmitted data (from 8 to 64 bytes), and the possibility of speed adjust of the data packet flow, as long as the transmission has already been started and the network nodes have already been synchronized. The arbitration field was maintained according to the classical CAN in 1 Mbps, but allowing the speed increase in the other message fields [4], [10].

According to the relevant information obtained from previous research about fault susceptibility in CAN and CAN-FD [11], [12], it is possible to model fault behaviors in the early design phases, thus contributing to a more efficient diagnosis process, and improving the performance of real-time applications, which is the focus of this work. Thus, considering as the central point the in-vehicle communication protocols, the present work contributes to providing a connection and interaction between test and design phases, modeling the negative impact of faults as non-functional requirements (NFR). Connecting these two phases, as presented and detailed in this work, is an advance compared to what is found in the literature, particularly in the application domain under concern. The application of the modeling techniques contributes to design more reliable systems, as handling performance degradation generated by faults in design phases is a promising approach.

The approach proposed combines NFR specification and aspect-oriented modeling strategy, allowing the modeling of a

Manuscript received 4 June 2019; revised 12 January 2020, 3 October 2020, and 11 February 2021; accepted 11 March 2021. Date of publication 6 April 2021; date of current version 8 July 2022. This work was supported in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001. The Associate Editor for this article was C. G. Panayiotou. *(Corresponding author: Alexandre Dos Santos Roque.)*

Alexandre Dos Santos Roque, Edison Pignaton De Freitas, and Carlos Eduardo Pereira are with the Research Group in Control, Automation and Robotics (GCAR), Electrical Engineering Department, Federal University of Rio Grande do Sul (UFRGS), Porto Alegre 90035190, Brazil (e-mail: as.roque@ufrgs.br).

Nasser Jazdi is with the Institute of Industrial Automation and Software Engineering, University of Stuttgart, 70550 Stuttgart, Germany.

Digital Object Identifier 10.1109/TITS.2021.3067552

runtime fault diagnostic mechanism to provide data about the performance of intra-vehicular control systems. The research problem is centered on the lack of specific fault handling approaches considering both the control system criticality and communication protocol fault susceptibility. In this direction, observing the gap in the literature concerning the handling of these problems, this work presents as the contribution, the combination of fault modeling in early-design phases with data obtained by a run-time mechanism which monitor the performance degradation in intra-vehicle networks. This proposal innovates compared to others found in the literature providing an alternative connection between design and test phases, which benefits the entire system development. In addition, the proposed approach focuses particularly on critical control systems, with hard real-time constraints. The present study deals with communication faults and the susceptibility of the current protocols to performance degradation. As an example of application to validate the proposal, a case study was conducted applying the diagnostic system in a real vehicular network with an active suspension control system under disturbances. This system has critical messages which must comply with strict timing constraints, affecting not only the comfort but the car drivability.

The proposed runtime mechanism is based on events that are generated and recorded in anomalous situations, allowing the correlation and subsequent analysis of the possible negative effects of performance oscillation in the control system. For the definition of tolerance limits, experiments that aim to observe the correct operation of the control system is necessary. Thus, an essential connection is made between system test and compliance, with the system design, in order to provide support for runtime anomaly detection. All the tests and experiments performed in this paper are supported by Vector CANoe/Analyzer hardware and software platform [13], widely used in the automotive industry, for analysis and control system development.

This paper is organized as follows: Section II presents related work about mechanisms for modeling, detection, and diagnosis in vehicular networks, also focusing on the communication protocols they address; Section III describes the proposed runtime diagnostic mechanism based on the modeling approach; Section IV presents the developed case study as proof of concept, applying the mechanism in a specific vehicular control system; Section V provides the assessment of the results obtained in the case study, while Section VI is provided conclusions with perspectives for future works.

II. Related Work

The central discussion of this section is about approaches focusing on fault analysis, vulnerabilities and diagnostic systems in CAN and CAN-FD protocols.

The real-time diagnostic of distributed vehicular control systems is a prominent and challenging area. In [14] is presented the integration of a CAN/OBD-II system, 3.5G mobile network, GPS, and a system to develop a technology for real-time diagnostics, but with focus on the high-level data. However, the focus on On-Board-Diagnose (OBD) systems and their limitations, lead to important gaps that must be addressed, mainly related to performance degradation and the negative effects in critical control systems.

The work [15] presents a method with a focus on the prediction of fault occurrence by analyzing system events related to control tasks, which are repeated at a determined frequency and time interval. Researches highlight that the fault diagnosis is a recurrent problem in CAN networks. The work [16] proposes an adaptive algorithm for fault diagnosis on CAN, designed for low-cost resource-constrained distributed embedded systems. The goal is the detection of faulty nodes, allowing new nodes entrance during a defined diagnostic cycle. Distributed control systems are the basis of critical automotive applications, and the runtime monitoring of control tasks is a recent challenge in the industrial area. In [17] it is presented an investigation about malicious attacks in industrial control systems, specific in the concern of cyber-physical systems, analyzing the effect on the performance metric jitter. Nevertheless, the approach lacks a specific analysis focused on the effect on the communication protocol.

In [18] it is presented a design of an approach linking security and dependable issues for future cybercars. The work presents a case study based on a steer-by-wire (SBW) application over the CAN network. The work emphasizes the early design analysis considering the impact in time constraints together with security primitives. Besides this important contribution, it lacks the specific handling associated with the disturbances on the performance that could not represent a problem yet. The registration of all performance data from ECUs for analysis is also considered as future research work.

Some recent works also focus on the analysis of fault impact on protocols as CAN, and in more robust protocols as CAN-FD. In [19] the authors propose an algorithm for error handling to replace the native CAN error handling. The methodology is evaluated using a steer-by-wire system of vehicles to analyze the effect of fault occurrences in the CAN protocol. The accelerated growth of the distributed systems makes the concern with the use of the communication buses increasingly relevant, leading to the development of other alternatives, safer and more reliable, as presented in [20], which focused the protocol CAN-FD. These alternatives take into account the greater data capacity that CAN-FD offers over the conventional CAN, but not free of faults that can degrade its performance. In the experiment reported in [21], the attention was focused on attacks in the communication network, attacks that could be some type of external communication or even failures in lost communications of the nodes themselves.

Problems related to fault identification and detection are also handled in recent researches. In [22] the authors handled problems about the intermittent connection (IC) of network CAN cables. The work proposes an IC fault localization algorithm, which utilizes the data link layer information of the CAN network for IC fault pattern analysis. Otherwise, in [23] the authors propose a monitor scheme based on a runtime verification method, which can verify selected properties of an automotive application. Both works have important contributions and are related to the present approach, but differently, they do not evaluate performance degradation over time and

the registration of the associated events with specific control system messages.

As presented in these researches, the intra-vehicular networks could generate massive data and in this scenario, the data analysis becomes a big challenge. About this issue, the work in [24] presents an offline approach to detect known and unknown faults in automotive systems. The approach focus on data records obtained from the in-vehicle network during road trials and an anomaly detection method is proposed to detect faults. The presented results report about anomalies in the multivariate time series which point the expert to potential faults. In [25] the focus is on feature degradation in fault-tolerant systems, addressing mixed criticality and mixed reliability in automotive systems. The work considers degradation of functional features, called fail-degraded features, and proposes a formal system model that contains the functional features of a vehicle and possible feature degradation. Although these studies highlight the importance of in-vehicle data analysis, techniques that correlate the performance degradation effect, in critical tasks, and at the protocol level, are not addressed.

Another important gap observed in recent researches is the lack of a link or connection between test and design phases. In [26] is highlighted that early fault detection is crucial to reduce the downtime and the negative impact on a wide variety of industrial applications, including the automotive system. The work emphasizes the problem of extraction features from incipient signals and the detection of anomalies, facts also handled in the present work, but focused in early fault modeling, runtime data analysis and events generation, for in the future with more consistent information, apply intelligent algorithms to address the faults. The survey [27] emphasizes the necessity of detection any kind of potential abnormalities and fault as early as possible, also implementing real-time systems in order to minimize the performance degradation avoiding dangerous situations. In this sense, Table I presents a summary of the main aspects and gaps in the literature.

The presented related work highlight the addressed research gaps as well as some other that still need to be addressed. Table I summarizes these discussions, highlighting in the last line the gap that the present study covers, specifically the "early fault analysis and modeling", considering the performance degradation in the CAN and CAN-FD communication protocols, proposing a mechanism to record such events. As emphasized in [16], [19], [21], and [26], fault modeling must be addressed early, and the present study aims to address this issue in an area that is continuously growing in complexity, i.e. the modern automotive distributed control systems.

Recent advances in vehicular networks lead to problems related to different interference, intentional or not, that causes sudden performance degradation. The present work combines different approaches focusing on early fault modeling to develop and test a runtime fault diagnostic mechanism for performance analysis, generating data about intra-vehicular network performance.

TABLE I
RELATED WORK SUMMARY

Ref.	Main Approach	Discussed Research Gap
[14]	Integration of different mobile systems with OBD and CAN	Exploration of real-time diagnostics in high level data
[15]	Redundancy Enabled Collaborative Recovery model	Fault isolation process, imprecise information
[16]	Fault diagnosis algorithm for CAN	Modeling faults in design phases
[17]	Investigation about malicious attacks in industrial control systems	Jitter analysis during real-time control system operation.
[18]	Security and dependable issues for future cybercars	Multi-core ECUs for error detection. Lacks of real-time performance analysis
[19]	Algorithm for effect analysis of faults in CAN-based networked control systems	Fault modeling and analysis in automotive systems
[20]	Tests comparing the greater data capacity of CAN-FD and CAN standard	Simulations tests / without communication and performance degradation analysis
[21]	Attacks in the communication network focusing on an IDS	The work lacks a fault analysis and modeling
[22]	Fault localization algorithm for CAN-based systems	It lacks performance and event registration
[23]	Monitor scheme based on a runtime verification method	It lacks performance and event registration
[24]	Offline analysis of in-vehicle network subsystems, with data recorded during road trials	Fault detection in automotive systems
[25]	Formal system model about critical automotive components	Analysis of degradation in functional features
[26]	Deep architecture for system downtime reduction	Early fault detection, anomaly detection
[27]	Survey of fault diagnosis techniques	Anomaly detection and performance analysis.
[This Work]	A runtime diagnostic mechanism supported by a fault modeling approach	**(Covered Gap) Early-modeling and Performance analysis in CAN/CAN-FD vehicular protocols with a fault diagnostic mechanism**

III. Proposed Runtime Diagnostic Mechanism for Vehicular Distributed Control Systems

According to related works analysis, it is possible to observe that there are gaps to be explored regarding fault modeling in design phases and also related to runtime diagnostics mechanisms. In order to clarify the types of faults and domains that could be handled with the present approach, Fig. 1a presents the fault classification according to [28] and Fig. 1b presents the fault model used to validate the present study.

Based on these research directions, this work proposes a Runtime Diagnostic Mechanism for performance monitoring and events registration in distributed vehicular control systems. This proposal is in line with previous works about fault modeling as non-functional requirements - NFR [29], [30] [31]. The present approach is based on the fault modeling process presented in [31], but in this work, a focus is given to a practical evaluation in a real in-vehicle networked control system and supported by a disturbance tool for fault injection. The previous work was focused only on the fault modeling and non-functional requirements specification, lacking a real study and evaluation, covered in this present paper.

The fault-tolerance concepts are associated with three domains (fault, error, and failure) [28]. Thus, in the present work, the faults in the physical context are addressed,

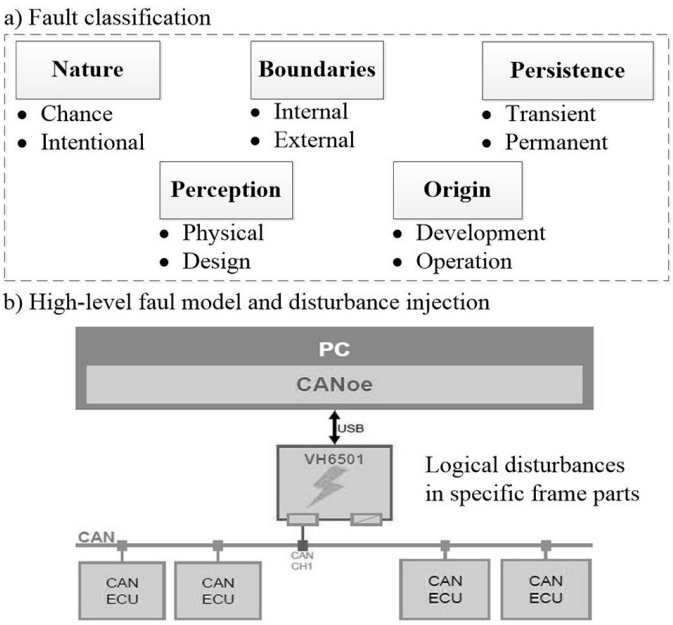

Fig. 1. Faults classification and fault model.

according to their persistence, that affects the communication protocols (through interference, in their internal and external limits), and by consequence, they can lead to generate errors and failures. The diagnostic mechanism considers the time domain in real-time control systems and can be applied to study the effect of any fault or disturbance that degrades the performance of the in-vehicle communication network, even those caused by malicious attacks. Recent researches are focused specifically in transient faults [11], [12] and the present study highlights the fault impact according to logical disturbances in specific frame parts and different network busloads (Fig. 1b).

The main point is that disturbance events generated by faults are registered and after analysing the behavior of these faults, they could be mapped supported by a framework for requirements specification, i.e. the extended version of RT-FRIDA presented in [29] and [31]. The RT-FRIDA framework is dedicated to identifying the system functional and non-functional requirements. Use case diagrams and templates are used to elicit these requirements. For this task, check-lists, lexicons, and conflict resolution rules are used. A link among classes, actors, and the use cases is created, and the NFR is visually represented in the class diagram. In the end, the source code of classes and aspects (i.e. skeletons for classes and aspects) is generated [32].

Corroborating with this modeling process, through the application of the framework, the designer can separately specify the non-functional timing and performance requirements in relation to degradation caused by faults. The approach allows the specialization of one additional domain of interest in the system detailing, emphasized in researches by the possibility of handling behavior and degradation generated by faults since early design phases. In this case, it is the responsibility of a designer to specify the requirements and to make this separation clear. The present approach contributes to this direction.

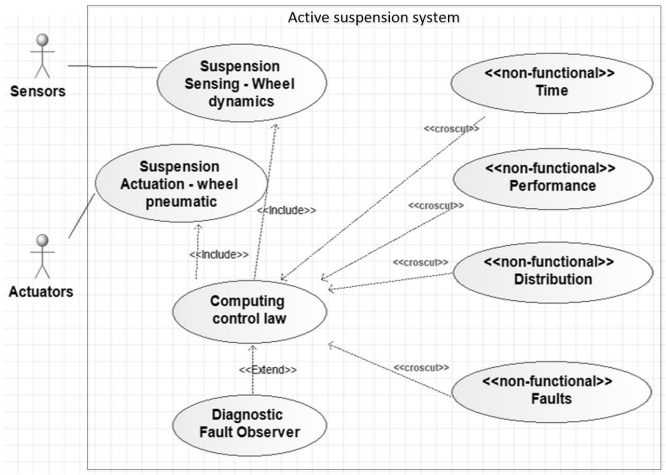

Fig. 2. Use Case Diagram of the Active Suspension System with the Diagnostic Mechanism. On the right side, the AOM stereotypes based on the RT-FRIDA framework.

A. Design of the Diagnostic Mechanism

The present work uses as basis an active suspension control system presented in [33]. This system is used in order to implement and evaluate the proposed approach in a real vehicular scenario. This control system was chosen because in [33] the control system modeling and performance test was already evaluated lacking a specific analysis regarding disturbances and faults. Another important reason is that in the present study the focus is only on the fault diagnostic mechanism, emphasizing the possibility of being incorporated into any vehicular control system design. The nature of disturbances comes from logical disruptions in frame parts that could simulate real fault occurrences.

After the study of the NFR related to performance degradation caused by faults and the RT-FRIDA application, a use case diagram was developed representing the context of study. The diagram serves as a representation of the start point of the diagnostic mechanism modeling process, which is presented in the Fig. 2. The main "use case" for the active suspension system is the "Computing control law" that has the hardware and logic implementation of the control system. The actors "sensors" and "actuators" interacts with the logic implementation of the "suspension actuation (wheel pneumatic)" and "suspension sensing" (wheel dynamics). In the diagram of Fig. 2, the entire design presented here is centered on the "Diagnostic Fault Observer" use case, which represents the fault diagnostic mechanism.

According to the RT-FRIDA framework [32], this task is performed by adding an use case with the ≪*nonfunctional*≫ stereotype for each nonfunctional requirement, followed by another ≪*crosscut*≫ stereotype for the representation of the relationship between the requirement and the affected use case. The second stereotype comes from the concepts of aspect orientation handled in the framework, which works the transversality of requirements that affect different parts of the system. The "non-functional" use-case has a different semantic in relation to the ordinary use case. Following this approach, it is possible to create an use case for each nonfunctional requirement and in order to make it clear, an use case can

represent each general classification item (example: time, performance, distribution, embedded and faults) that encompasses specific metrics, as example, Average Jitter and Difference Jitter). Besides that, the developed diagram in Fig. 2 considers a specific case study (active suspension control system), the fault diagnostic mechanism could be incorporated into any control system as long as following the approach presented in this work. For example, it is possible to consider another control system, in the automotive area or even in other industry domain, applying the approach to analyze the control system behavior under faults and allowing the run-time diagnosis, linking the data obtained during tests to improve the control system design cyclically.

Thus, in order to detail the proposed design, UML diagrams are presented to elucidate how the fault diagnostic could be applied characterizing a reference model for different languages and paradigms used in the embedded vehicular control system development. The class diagram and the state transition diagram are supported by stereotypes from the UML MARTE profile [34] because it is a standard reference for embedded and real-time system design. The choice of these diagrams follows the precepts that this design can be applied in the future to different development platforms in the automotive area. Development platforms could use structured, object-oriented or event-oriented paradigm. As the case study is centered in the Vector tools [13], during the tests and practical implementation, as an example, the event-oriented paradigm was applied. The engineer could understand the model and choose or use the best tool for her/his project. In this study, it is applied the VECTOR CANoe software for test and development, a tool that allows the abstraction of complex concepts of languages and transforms all the control processing that occurs in the communication network in events triggered when conditions are satisfied. Thus, based on the design overview presented in Fig. 2, Fig. 3 presents the developed class diagram for the diagnostic mechanism detailing the specific development parts.

For the representation of features and system behavior, the UML MARTE profile provides different stereotypes that can be used to annotate the model. According to UML MARTE the stereotype ≪*deviceResource*≫ represents an external device that can be scheduled or required by the control system. The stereotype ≪*allocated*≫ represents resource allocation and is usually applied to any element that maintains the allocation and utilization relationship. Thus, these stereotypes are used to represent the resources used by the active suspension control system plant, which has a set of sensors and the actuators. Another stereotype used is ≪*computingResource*≫ which represents any virtual or physical processing resource that stores and runs programs. In this case, the stereotype represents the ECU, which performs the computation of the control law and also requires other resources, as sensor data messages. The ≪*saSharedResource*≫ stereotype is related to shared resource and communication analysis, commonly applied in real-time systems. This stereotype is used in conjunction with ≪*computingResource*≫ to represent the diagnostic system, which in this study was aggregated into the network as an extra processing node.

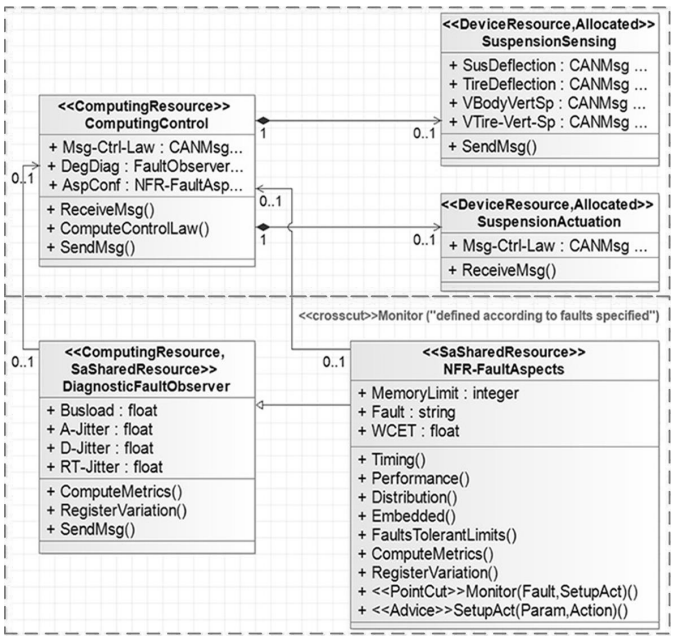

Fig. 3. Designed Class diagram with the diagnostic mechanism and the nonfunctional requirements.

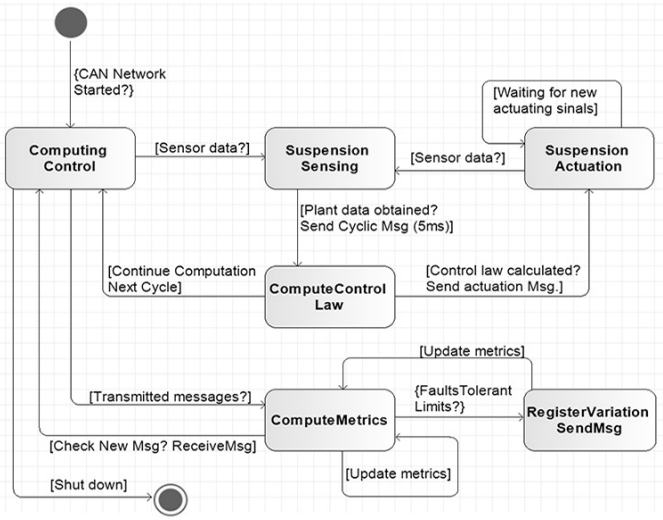

Fig. 4. State transition diagram for the DiagnosticFaultObserver mechanism.

The design emphasizes diagnostic mechanism for performance degradation monitoring generated by faults, as well as all aspects that represent the NFR handling that affect the control system. In addition, the state transition diagram also was developed to represent more details of system components, since it contributes to show the system state changes over time, helping the engineer in the understanding of this design when developed also on the basis of others paradigms, such as the structured and the event-oriented. Fig. 4 shows the developed state transition diagram showing the link between elements in the control system and how the network information is captured for performance and diagnostic analysis. The state diagram has the goal of describe the lyfe-cycle of the "Diagnostic Fault Observer", represented in a specific use case in Fig. 2 and detailed by a class in Fig. 3.

During the development of these diagrams, the designer defines the system representation, which can later be adjusted

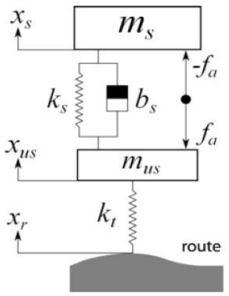

Fig. 5. Suspension plant based on the Quarter-Car model.

according to details of the defined programming language and development platform. Since in this study the used equipment was the VECTOR CANoe and its CAPL language (C++ based), in the validation step the specified design is coded based in the event-oriented paradigm. Such a decision does not affect the results that are presented since the design must be a reference for development and not restricted to a specific language. Another point to justify such a decision is the wide adoption of the chosen platform by the automotive industry, which is the one used in present application scenario. The following section presents the proposed design applied in the development of a Runtime Diagnostic Mechanism to monitor and notify the performance degradation of distributed vehicular control systems.

IV. Case Study of the Mechanism Application in a Critical Vehicular Control System

The evaluation of the proposed mechanism was made incorporating the design in an specific vehicular control system. As mentioned before the control system under test is an active suspension control system that was developed based on the previous works reported in [35], [36] and [33] and according to the quarter-car model. Fig. 5 illustrates this suspension (plant) model. The plant model is described by specific parameters presented in Table II.

The undamped mass represents the set composed of tire, wheel, suspension system and all auxiliary devices that form the link between the chassis and the ground. This model is based on [33], considering a LTI (Linear Time Invariant) model.

Thus, from Newtonian mechanics, it is possible to obtain the following equations that represent the dynamics of the system.

$$m_s x_s = -k_s(x_s - x_{us}) - b_s(x_s - x_{us}) - f_a \quad (1)$$

$$m_{us} x_{us} = k_s(x_s - x_{us}) + b_s(x_s - x_{us}) + k_t(x_r - x_{us}) + f_a \quad (2)$$

According to [35], different transfer functions could be applied to evaluate the performance of an active suspension system, being the suspension deflection *Xdef* one of them and the one considered in this present study. This transfer function represents a restriction on actuator deflection. The Suspension deflection is defined as the difference between the vehicle's body vertical position *Xs* and the suspension set baseline position *Xus*. The suspension deflection is defined by:

$$Xdef = X_s - X_{us} \quad (3)$$

TABLE II
Description of the Suspension System Parameters

Symbol	Description	Unit
m_s	Automobile body mass (1/4)	*kg*
m_{us}	Suspension, wheel and tire mass.	*kg*
x_s	Vehicle body vertical position.	*m*
x_{us}	Suspension assembly vertical position.	*m*
x_r	Unevenness caused by the path (ground).	*m*
x_{def}	Suspension deflection.	*m*
k_s	Suspension spring constant.	*N/m*
k_t	Tire model spring constant.	*N/m*
b_s	Suspension damping constant.	*N*s/m*
f_a	Actuator applied force (control signal).	*N*

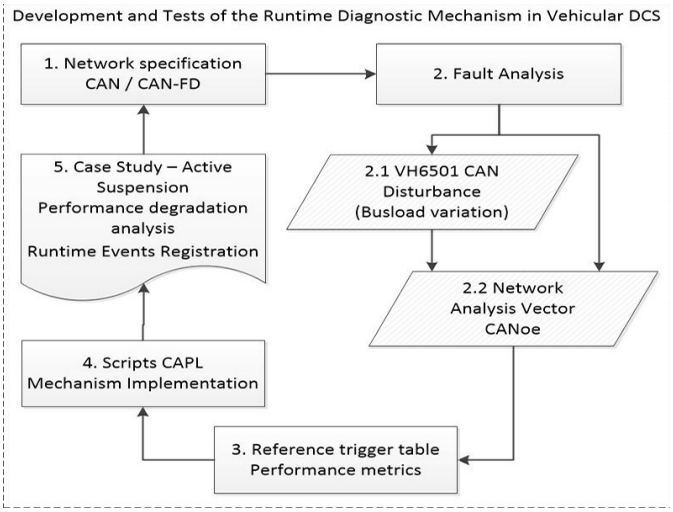

Fig. 6. Mechanism development and test sequence.

An important feature to note is that the control technique applied to the plant dynamics of the active suspension control system is based on the research presented in [33] (Controller: State feedback based on the methodology of data sampled), so details about this process are omitted in this step. The focus of the present study is, based on the dynamics of a previously defined control plan, to model the communication dynamics of the control system together with the performance diagnostic mechanism.

The active suspension control system represents an specific test scenario with cyclic critical messages and according to a modeled control plant. On the other hand, despite all complexity in the physical plant, the main issue in this development is the communication with the mechanism incorporation and evaluation in both CAN/CAN-FD vehicular networks.

A. Network Specification and Test Procedure

According to the case study scenario, to clarify the development of the proposed design presented in the Section IV, a sequence of steps was defined representing the materials and methods applied in the experiments. Fig. 6 presents a representation of this development.

In the first step ("1. Network specification CAN/CAN-FD") was defined the network topology (BUS topology), the communication protocols configured for the tests (CAN 1 Mbps

TABLE III
NODES CONFIGURATION

Node	Function	Data / TX–RX messages
Sensor	Mass-spring-damper assemblies. Parameters of the suspension system.	MSG–Body–Vert–Speed MSG–Susp–Deflection MSG–Tire–Deflection
Actuator	Adjustments in the vertical position of the suspension assembly.	MSG–Susp–Set–Vert–Speed
Controller	Computation of the appropriate levels of suspension parameters. Control Law.	MSG–Control–Law

and CAN-FD 1-4 Mbps) and the messages that composes the distributed control system. Table III shows the messages and details of the configuration/function of each node in the active suspension control system.

The messages defined in the Table III are cyclically transmitted (control law with 5ms period), and represents the standard operating conditions of this control system. In accordance with these node parameters, the CAN-FD network was configured to operate with 1 and 4 MBps during the experiments and with five data packets of 8 bytes each. In the experiments, the hardware used for the node implementation was the Vector VN8900 with VN8970 CAN/CAN-FD modules. For the tests in standard CAN, the configuration was 1 Mbps in CAN-FD with 1 Mbps in arbitrary frame parts (standard protocol) and until 4 Mbps in the payload. The controller node is responsible for the computation and represents the link between the controller node and the nodes from the plant behavior (sensors and actuators). Different payloads were sent through the CAN and CAN-FD network to increase the busload during the experiments. This decision was adopted in order to evaluate the diagnostic mechanism and the system behavior in critical situations and also with high bandwidth usage.

The second step ("2. Fault Analysis") represents the disturbance injection in the network that was made using an specific trigger defined in the Vector VH6501 and also with an extra node only inserted to generate traffic. The network configuration and analysis is done by the Vector CANoe software. The decision to use these tools was based on their wide adoption in the automotive industry and for presenting possibilities of network compliance tests close to real scenarios. The tools applied in this study were:

- CANoe - CAPL implementation/network configuration.
- VN8910/VN8970 - Hardware configuration for nodes.
- VH6501 - CAN/CAN-FD disturbance injection hardware.

More details about this configuration are presented in the next subsection.

The third step ("3. Reference trigger table") represents tables developed and filled out with data obtained in the previous tests. In order to check the typical delays during the usual operation and without interference or disturbances, several preliminary tests were performed and according to these data, a table was filled out measuring the network in four busload scenarios (12% - only the control system, 30%, 50% and 80% with the control system and traffic generated by software). Tables IV and V show the data obtained and the

TABLE IV
REFERENCE TABLE FOR CAN PROTOCOL TESTS

Metric/Trigger	Bus 12%	Bus 30%	Bus 50%	Bus 80%
Difference Jitter	190	1014	1997	4970
Ref Trig1	200	1000	2000	5000
Average Jitter	45,32	111,52	235,32	790
Ref Trig2 (+25%)	60	140	294	850

triggers defined for CAN and CAN-FD tests and all values are in microseconds. The reference for the difference jitter trigger was defined based on the peak value of the period (rounding). For the average jitter, it was defined as reference a value above around 25%. These decisions were according to the oscillations registered during faults and disturbances analyzes in recent researches [11], [12]. It is important to highlight that these tables represent an application example (based on the case study) of how the control system operating limits could be defined and analyzed during test phases. The same approach could be applied in other scenarios, analyzing if the real-time control system could provide deterministic guarantees under faults, interferences and in different busloads.

Based on this information, the next steps ("4. Scripts CAPL" and "5. Case Study tests") represent the implementation of the runtime performance diagnostic mechanism and validation tests respectively. In the implementation, the diagnostic mechanism uses the information from Tables IV and V to observe the network, storing communication logs and generating alerts when these triggers are activated.

Metrics for network performance evaluation are essential for test phases and typically the jitter is used to evaluate the oscillation time between messages received and transmitted. The jitter is calculated based on the transmission time of a message on the bus [37]. For example, a CAN message with 8 bytes takes 130 us to be transmitted at 1 Mbps with BitStuff of 5 bits. This is the worst-case response time of a message on the bus.

The calculation is automated in the Vector CANoe software, which provides the difference time between messages. Thus, for the calculations it was used as the basis the Average Jitter (the standard deviation in the measure of average message transmission time) and Difference Jitter (subtracting the best-case transmission time from the worst-case transmission time) [38]. However, the average jitter is instantaneously calculated and updated according to the evolution of the system.

Following the equation of the runtime average jitter updated during the network monitoring, and based on the approach presented in [38]:

$$InstantAvgJit = \sqrt{\sum \frac{(JM_{[i]} - MT_A)^2}{nMsg}} \tag{4}$$

where,

$JM_{[i]}$ is the jitter in a given cycle belonging to the set measured; The jitter of each message is calculated according to the "DiffTime" function present in the software CANoe that represents the difference time of a transmitted cyclic message. Each variation in this time is stored in an array, allowing calculation based on the network monitoring time.

TABLE V
REFERENCE TABLE FOR CAN-FD PROTOCOL TESTS

Metric/Trigger	Bus 12%	Bus 30%	Bus 50%	Bus 80%
Difference Jitter	185,45	297,56	1057	4680
Ref Trig1	190	300	1060	5000
Average Jitter	20,34	49,50	95,62	673,21
Ref Trig2 (+25%)	25	62	120	842

MT_A is the arithmetic average up to the specific time;
$nMsg$ the total number of a given message.

It is important to highlight that the calculation must be associated and registered for each specific critical message that composes the distributed control system.

An algorithm was developed to activate the performance triggers according to previously defined range limits, allowing the real-time diagnosis of the control system. In the following, the pseudo code of the developed algorithm is presented, which computes the performance metrics in runtime (Average Jitter and Difference Jitter).

Algorithm 1 RT-Jitter Algorithm

```
1: procedure CRITICALMSGMONITOR(MESSAGESID)
2:     RangeLimit ← PercentTolerance for each message
3:     TimeCtrlLaw ← Time Interval in ECU Control
4:     Jit ← Average Instant Jitter Calculation
5:     loop: for each critical MessageID
6:     if Jit > RangeLimit then
7:         Register Log Event for the Node
8:         TrigJit ← LogTimeEvent
9:         MsgID ← MessageID
10:        SendMsgDiagnostic()
11:    checkBufferMemory()
12:    updateCurrentRTJitter for the MsgID
13:    loop
```

The present technique allows the system diagnosis under fault and interference by registering all disturbances events, associating the node and the control system under observation. It is important to highlight that the focus is to monitor critical real-time control systems with hard timing constraints, typically with cycle time between 1 and 20 milliseconds. The current hardware used for tests has local memory buffers and this feature need also to be considered in future implementations in order to support the high frequency of events that could be registered. The algorithm also provides the possibility of sending diagnostic messages, allowing high-level application development, for example, to generate notifications and also send diagnostic data to cloud systems.

The stored events allow the engineer the observation of how the control system operates during different critical situations, with high busload and also with faults and disturbances. In the same way, it is possible to check the performance of communication protocols in the same situations, indicating the limits and the critical parameters for the correct operation in critical control systems.

B. Tests With the VH6501 Can Disturbance Interface

The Vector VH6501 CAN disturbance interface has capabilities of digital disturbances generation in CAN networks. The tool allows the observation of the control system behavior under performance degradation, with the possibility of generating disruptions in specific CAN frame parts.

The VH6501 tool has at least 9 logical disturbance presets, called within the tool by "Desktops". Each preset differs only in the disturbance trigger type, most of which are focused on triggers at specific points in the CAN frame. Although it is possible to use the trigger panel for each of the test desktops, for the purpose of automating the performed experiments in this study, all disturbance tests were triggered by timers configured within the script CAPL of disturbance injection.

Thus, for the tests, three disturbances desktops were defined "Software Trigger", "Frame Trigger" and "Missing Bit Trigger", considered more complete because generates the common errors that occur during typical faults in communication protocols. According to these configuration it is possible to configure a variety of injection sequences increasing the disturbance level in the network. These desktops immediately sends a bit sequence or a frame sequence in the CAN network. The effects of these configuration leads to the following errors in the protocol:

- Stuff Bit Error - A dominant bit of 2 us or 1 us is sent into the bus, so the CAN interfaces understand that it is a SOF (start of frame), but nothing is sent in sequence.
- FrameDisturb - This option allows the sending of a disturbance in a specific frame, using its identifier as parameter, but without data (DLC = 0).
- Disturb Ack Slot and Ack Delimiter - A trigger is configured for a message based on its identifier, having its position defined in the CRC delimiter. Thus, a disturbance is generated in the next bit of Ack slot, sending a recessive level rather than a dominant one.
- IFS Disturb - trigger in the second bit of the IFS inter frame space. A message of DLC = 0 is sent without respecting the space of inter frames generating error frames and increasing the bursts counter.
- Disturb CRC Delimiter - Trigger position in the CRC field of the frame. One-bit size disturbance sequences are generated in the CRC Delimiter field by sending a dominant rather than a recessive level.

Due to the large amount of presets in the VH6501 tool, there are other disturbances types that were not considered at the moment. In the present experiment the disturbance injection was made in two sequences, one with the "Software Trigger" and another with the union of the "Frame Trigger" and "Missing Bit Trigger" desktops. It is important to note that the CAPL test scripts were coded for the present study because the tool only presents example codes that should be adapted to each application. Fig. 7 presents the network topology configured in the CANoe software.

The Fig. 7 shows the "ECUs Susp-Contr" (Controller) and "Susp-ECU" (Plant with Sensors and Actuators) that represent the three nodes of the Active Suspension control system. The "ECU-FTObserver" has the proposed diagnostic mechanism implemented in CAPL code, and ECU "TG" represents the codes for traffic generation during the experiments, in order to increase the busload for all test scenarios. The last node

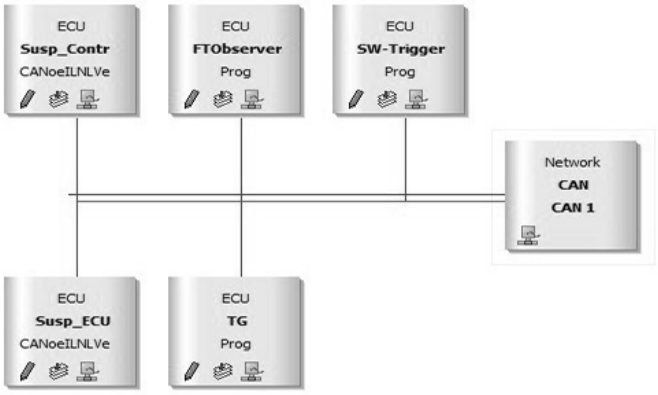

Fig. 7. Network configuration with the disturbance node.

is the "SW-Trigger" desktop from VH6501 tool, where the disturbances are injected in the network.

In order to evaluate the diagnostic mechanism in both CAN and CAN-FD protocols, as specified in Tables IV and V, the tests were performed in the same scenarios, with 12%, 30%, 50% and 80% of busload. The tests have the goal of applying the modeled runtime fault diagnostic mechanism, observing in a real in-vehicle control scenario, the possibilities of disturbance detection and events registration. The fault injection with the VH6501 hardware serve as an example of disturbances that degrade the control system.

Fig. 8a and 8b show the graphs about tests with disturbance injection with 12% of Busload (only the control system). The Fig. 8c and 8d show the graph with 30% of busload (control system and traffic messages).

In the same way, the Fig. 9 presents the tests performed with the mechanism application in higher busloads, 50% in Fig. 9a and 9b and 80% in Fig. 9c and 9d.

Fig. 9a shows that during the experiment, due to the amount of errors reach 255, the bus has changed to the BussOff state not completing the analysis period. This situation occurs due to the susceptibility of the standard CAN to disturbances, fact evidenced during the diagnostic mechanism application. After this observation the amount of disturbance injection was reduced to 50% of the repetitions (only on CAN protocol), adjusting the VH6501 in order to complete the experiments.

The oscillation in the "Instant Average Jitter" depends on the number of disturbance repetitions configured in parameters of the VH6501 tool, and also on the current bus traffic. The oscillation (for example, in Fig. 9c - CAN with 80%) occurs because in the CAN protocol the disturbance effect is higher, as observed the Bussoff state before, thus, the hardware was adjusted to stop the disturbances. The adjust occurs after 64 seconds for 5 seconds and after that continues normally. However, it does not affect the results because it only represents an oscillation in the disturbance injection showing that the run-time mechanism could correctly observe and detect the performance oscillation in the in-vehicle network.

Other important information obtained during these tests is the registered number of trigger events, representing the capabilities of the fault diagnosis mechanism in different scenarios, also analyzing the fault effect in distributed control systems. The triggers allows the analysis of how much time the control system is under performance degradation, because it is stored the time in which an event is activated and when it is deactivated. All disturbances events are stored in logs for further analysis and according performance metrics previously defined during the control system observation under normal operation. The next section shows more details about the results analysis in all test scenarios, also presenting an analysis of the mechanism application in CAN and the emergent CAN-FD protocol for distributed vehicular control systems.

TABLE VI
RESULTS DURING CAN PROTOCOL TESTS AND FIRST DISTURBANCE SEQUENCE

Metric/Events	Bus 12%	Bus 30%	Bus 50%	Bus 80%
Difference Jitter	5011	5656	16271	19091
Events Trig1	35	74	46	3019
Average Jitter	159,91	182,66	256,17	1728,93
Events Trig2	2	3	5	1

TABLE VII
RESULTS DURING CAN-FD PROTOCOL TESTS AND FIRST DISTURBANCE SEQUENCE

Metric/Events	Bus 12%	Bus 30%	Bus 50%	Bus 80%
Difference Jitter	4993	5890	6383	21665
Events Trig1	44	56	135	3996
Average Jitter	123,79	131,76	174,97	2613,47
Events Trig2	1	1	1	1

V. RESULTS ANALYSIS

The proposed diagnostic mechanism was evaluated in different busload scenarios using the VH6501 CAN disturbance interface, based on the network configuration for a distributed control system (Active Suspension Control System), in order to analyze the performance on CAN and CAN-FD protocols. The tests were conducted according to the steps presented on Fig. 6 and the references Table IV and V. The analysis evaluate the control system using the Average Jitter and Difference Jitter metrics, activating triggers in each case, and observing the disturbance impact in the communication protocols. The triggers were activated according to the reference data obtained during previous experiments, allowing the registration of when and how the disturbance degrade the control system performance. It is important to highlight that the runtime diagnostic mechanism generate a large amount of data, and because of this issue, the time sample chosen was 120 seconds, just to observe the applicability of this approach. The Table VI and VII presents the results obtained during all experiments in CAN and CAN-FD protocol. The metrics are in microseconds and the events are specified in registered number of occurrences.

In Fig. 10 and Fig. 11 graphs are presented summarizing the performance of the proposed mechanism in both analyzed protocols, highlighting the two metrics used for the detection triggers. These graphs present a comparison between the delay registered in the analyzed time sample, using the two disturbances injections sequences (Frame/Missing Bit and Software Trigger desktops from VH6501) in both CAN / CAN-FD communication protocols.

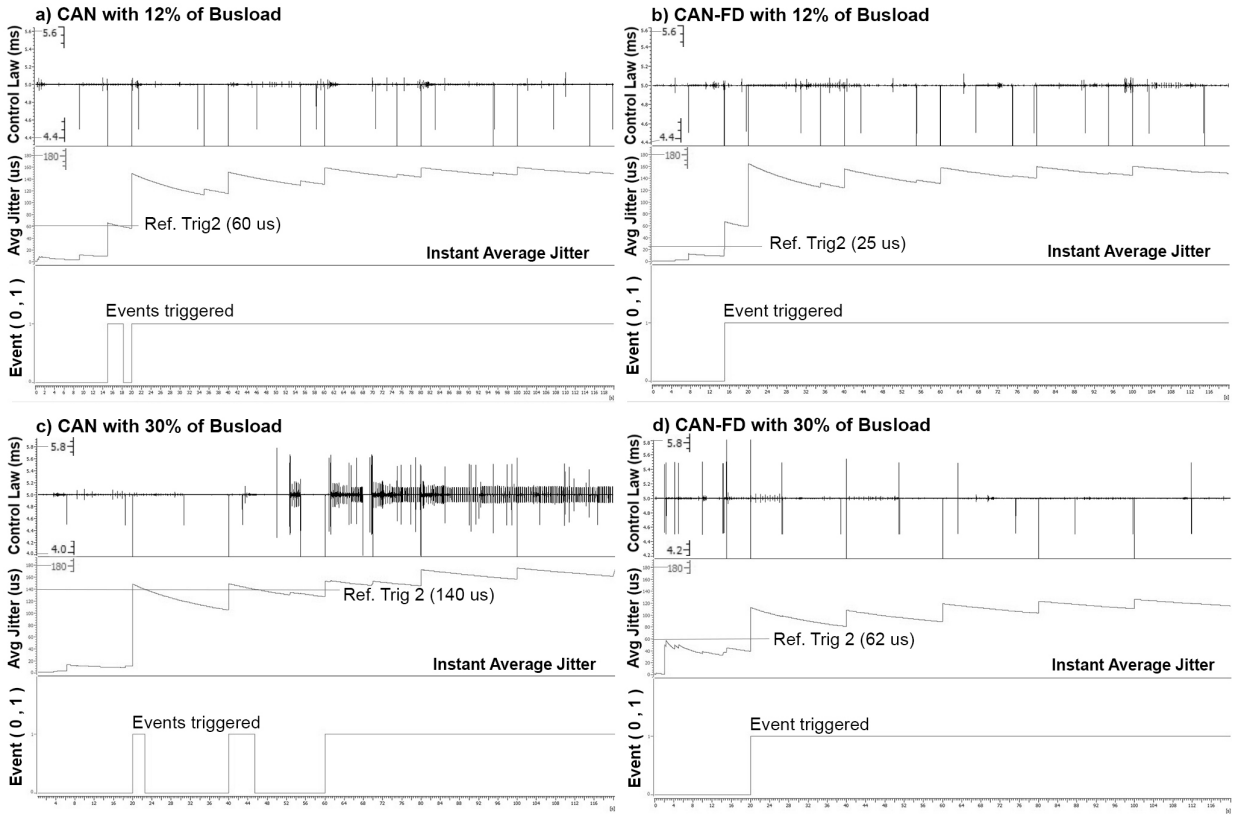

Fig. 8. Tests of the diagnostic mechanism performed in CAN and CAN-FD - 12% and 30% of busload.

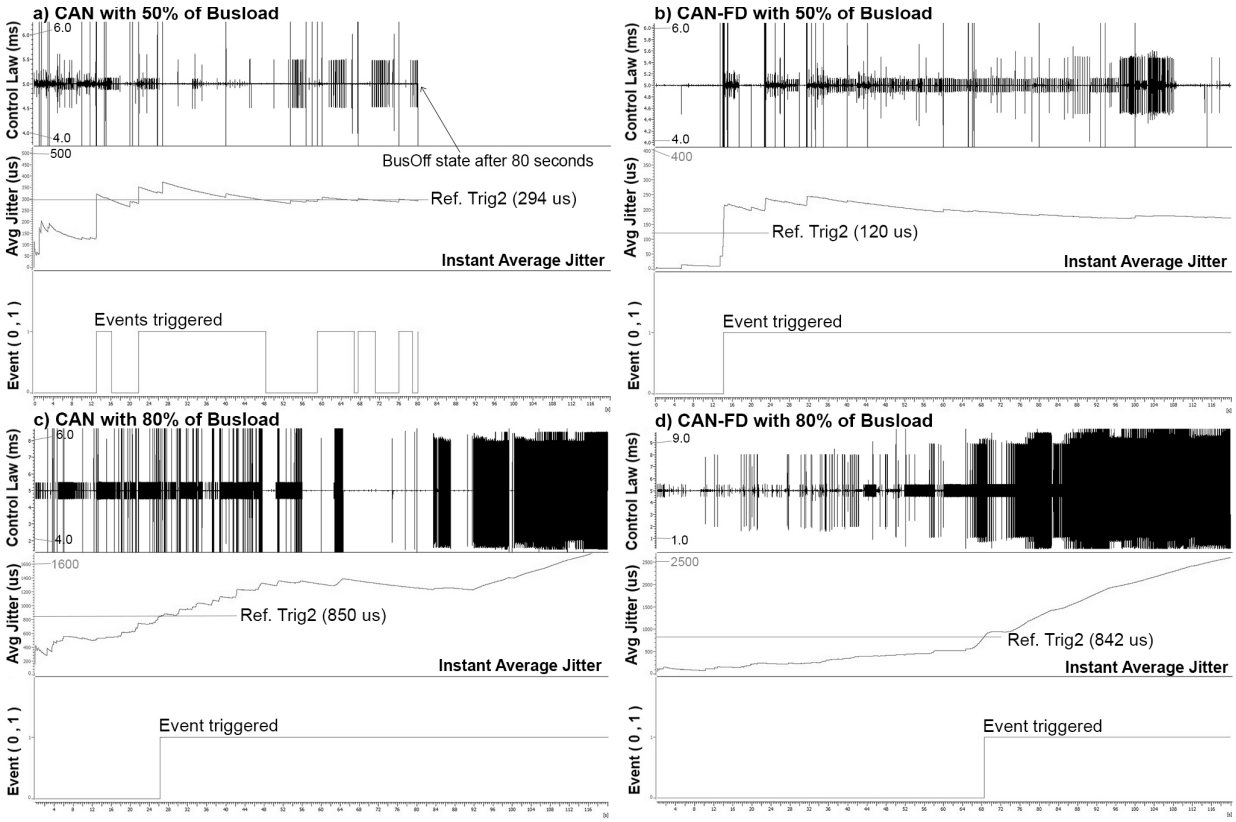

Fig. 9. Tests of the diagnostic mechanism performed in CAN and CAN-FD - 50% and 80% of busload.

According to Table VI and VII and observing Fig. 10 and 11 it is possible to evidence the applicability of the runtime diagnostic mechanism, checking the amount of events related to disturbances and performance degradation. During the analyzed time sample, the trigger 1 generates a bigger number of events because it is associated with delay peaks

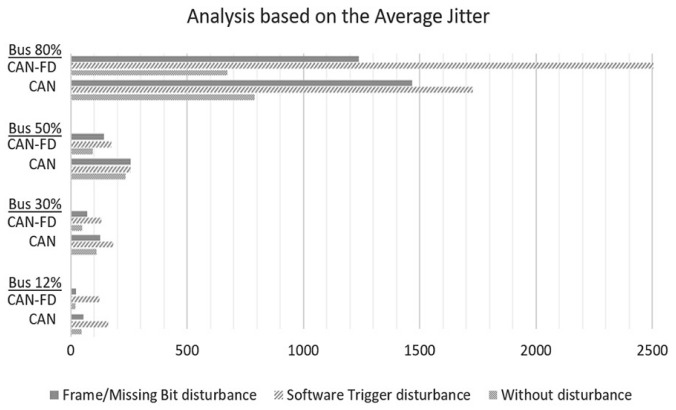

Fig. 10. Performance according to the Average Jitter metric in all test scenarios.

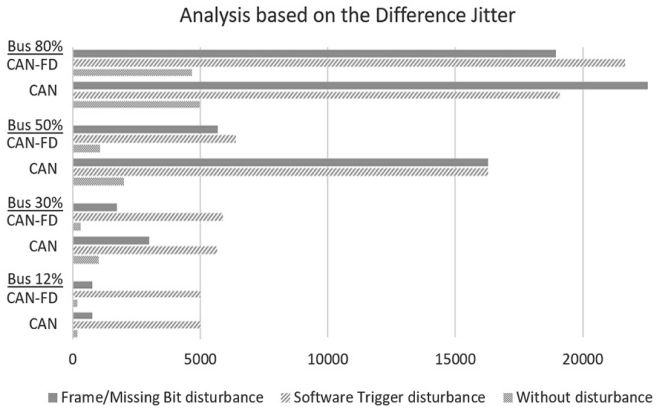

Fig. 11. Performance according to the Difference Jitter metric in all test scenarios.

between the control system messages (difference jitter metric). On the other hand, the trigger 2 is associated with the average delay between control messages (average jitter) and the events registration depends on the analyzed time sample, the protocol, and the amount of disturbance injection.

Trigger 1 indicates sudden performance disturbances in the control system allowing the verification of possible fault situation in the future (for example, with AI algorithm analysis), because with each registered event, it is possible the correlation between the nodes and the communication messages. Trigger 2 indicates a sequence of disturbances and a bigger time of performance degradation that could be analyses as an imminent fault situation. This fact was evidenced during the tests, because during the disturbance injection in standard CAN protocol with 50% of busload, after a sequence of trigger 2 events (average jitter reference) the bus reach to the "busoff state" requiring the CAN network reboot (Fig. 9a. At this point, it is important to highlight the need of adjustment in the sequence of disturbances using the VH6501 hardware, from 20 to 10 repetitions in order to complete the tests. Thus, the results presented in Table VI and VII, in the columns Bus 50% and Bus 80% of CAN standard, are with 10 repetitions in the disturbance configuration. During tests in the CAN-FD protocol, all tests were completed with 20 repetitions in the hardware configuration.

The results show the possibilities of disturbance detection with the proposed mechanism and that the CAN-FD could be more tolerant to disturbances. Except in the tests with 80% of busload, in the other scenarios, the average jitter in CAN-FD was lower than in the standard CAN protocol. The reason for this exception is because the CAN protocol does not tolerate the parameter of 20 repetitions on disturbance injections, leading the bus to the offline state, fact observed in Figure 9a. Thus, during the fault injection with 50% and 80% of busload, the disturbance injection was the double in the CAN-FD.

Based on these results, it is emphasized the applicability of the present approach for early fault modeling and NFR specification in vehicular distributed control systems. Relevant related work [15] proposes a collaborative recovery model for automotive systems, applying an event-triggered allocation model for the efficient allocation of critical resources (only on CAN protocol). The present work differs mainly in the fact that focus on fault analysis in network communication, with fault injection, covering both CAN and CAN-FD protocols. The analysis contributes to requirements specification of faults, handling the performance degradation observed in critical control systems. The approach here presented could solve similar problems addressed in [15], also contributing to timely activate recovery and redundant mechanisms. With the presented runtime diagnostic mechanism the obtained data generated during fault injection tests, serves as a reference for the continuous improvement of fault tolerance mechanisms. In addition, the generated data can be used to power artificial intelligence systems, observing when degradation in the performance of the vehicular network may be critical. Monitoring and diagnosing vehicular networks is an increasingly difficult task, and the present work presents a contribution to push forward the state-of-the-art in this area.

Another important related work is reported in [26], which focuses on early fault detection and analysis supported by deep neural networks. The present work is different as it generates an important database obtained from the tests of fault injection and focused on communication protocols. Tests with the diagnostic mechanism generate information about the in-vehicle network and the control system. This information is represented by features based on performance metrics, that can be useful for the improvement of fault detection models, including those presented in [26]. The present work is complementary to [26], as it can be an initial data source before the DNN application.

The performed experiments show a practical evaluation of how the present approach contributes to fault modeling in real-time control systems, representing an alternative compared to related works discussed in Section II. Most approaches do not consider a runtime and continuous analysis, linking test with design phases, associating the performance degradation with potential sources of faults and interference.

The adoption of the proposed approach could be done by following the design presented in Sections III and IV adapting it to any other in-vehicle distributed control system. The results show the benefits by comparing the previous fault injection analysis with the present approach based on performance metrics and the event registration. The data obtained during the

tests are used to continuously define and update new specific fault requirements (NFR) as mentioned in [31], which is the basis for the fault modeling process. The benefits of the present approach compared to related works are emphasized by presenting a specific case study of fault modeling (active suspension control system), describing how to model the process connecting the test with the modeling phases supported by the runtime diagnostic mechanism presented. The application and replication of this study will contribute to the reliability of critical automotive control systems since it will allow a more proactive fault diagnosis, better requirements specification that deals with faults, and especially, the runtime observation of the performance variability in the communication protocols.

VI. Conclusion

According to recent researches, the present work proposes a runtime performance diagnostic mechanism for intra-vehicular networks, focusing on the possibility of early fault modeling and the registration of events according to performance metrics. The event registration allows the analysis of a specific control system during a monitoring time, and for this task, it was configured a vehicular network with specific nodes for an Active Suspension control system. The approach focuses on a "runtime diagnostic observer" mechanism that is based on previous tests in the network, establishing performance limits for triggers associated with the registration of events. These events represent performance degradation in the control system under analysis. The work is evaluated in four busload scenarios and on both CAN and CAN-FD protocols.

According to the results, the registered peaks using the difference jitter metric tend to be slightly higher in the CAN protocol, but most of the time at the same limits. The main difference is that in the CAN protocol the frequency of these peaks is higher, a fact which raises the average jitter. On the other hand, it is possible to observe the best performance of CAN-FD with the average jitter metric. With 12%, 30% and 50% of busload the average jitter was 22,58%, 27,86% and 31,69% higher in CAN. In the test with 80% of busload the average jitter was 33,84% higher than in CAN-FD, but notice that with 50% and 80% of busload, the disturbance injection using the VH6501 hardware was double.

Future works go into the direction of considering the use of storage as an extra ECU in the vehicular network or a secure internet connection to send data to cloud-based systems. The obtained results contribute to generating data for intelligent and proactive diagnostic algorithm applications, for example, applying artificial intelligence techniques.

Acknowledgment

The authors would like to thank Vector Informatik GmbH, Stuttgart-Germany, for providing the Vector CANoe license and the VH6501 tool that were applied to perform the experiments and analysis of this work.

References

[1] W. Zeng, M. A. S. Khalid, and S. Chowdhury, "In-vehicle networks outlook: Achievements and challenges," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 3, pp. 1552–1571, 3rd Quart., 2016.

[2] L. L. Bello, R. Mariani, S. Mubeen, and S. Saponara, "Recent advances and trends in on-board embedded and networked automotive systems," *IEEE Trans. Ind. Informat.*, vol. 15, no. 2, pp. 1038–1051, Feb. 2019.

[3] S. Tuohy, M. Glavin, C. Hughes, E. Jones, M. Trivedi, and L. Kilmartin, "Intra-vehicle networks: A review," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 534–545, Apr. 2015.

[4] N. Navet and F. Simonot-Lion, "In-vehicle communication networks—A historical perspective and review," in *Industrial Communication Technology Handbook*, vol. 96. Boca Raton, FL, USA: CRC Press, 2013.

[5] T. Piper, S. Winter, O. Schwahn, S. Bidarahalli, and N. Suri, "Mitigating timing error propagation in mixed-criticality automotive systems," in *Proc. IEEE 18th Int. Symp. Real-Time Distrib. Comput.*, Apr. 2015, pp. 102–109.

[6] S. Woo, H. Jin Jo, and D. Hoon Lee, "A practical wireless attack on the connected car and security protocol for in-vehicle CAN," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 2, pp. 993–1006, Apr. 2015.

[7] J. Liu, S. Zhang, W. Sun, and Y. Shi, "In-vehicle network attacks and countermeasures: Challenges and future directions," *IEEE Netw.*, vol. 31, no. 5, pp. 50–58, 2017.

[8] P. F. Souto, P. Portugal, and F. Vasques, "Reliability evaluation of broadcast protocols for FlexRay," *IEEE Trans. Veh. Technol.*, vol. 65, no. 2, pp. 525–541, Feb. 2016.

[9] A. Hafeez, H. Malik, O. Avatefipour, P. R. Rongali, and S. Zehra, "Comparative study of CAN-BUS and FlexRay protocols for in-vehicle communication," SAE Technical Paper, 2017.

[10] F. Hartwich and R. Bosch, "CAN with flexible data-rate," in *Proc. iCC*. Citeseer, 2012, pp. 1–9.

[11] A. S. Roque, D. Pohren, T. J. Michelin, C. E. Pereira, and E. P. Freitas, "EFT fault impact analysis on performance of critical tasks in intravehicular networks," *IEEE Trans. Electromagn. Compat.*, vol. 59, no. 5, pp. 1415–1423, Oct. 2017.

[12] D. H. Pohren, A. D. S. Roque, T. A. I. Kranz, E. P. de Freitas, and C. E. Pereira, "An analysis of the impact of transient faults on the performance of the CAN-FD protocol," *IEEE Trans. Ind. Electron.*, vol. 67, no. 3, pp. 2440–2449, Mar. 2020.

[13] VECTOR Informatik GmbH. (2019). *Product Information Canoe/Cananalyser*. Accessed: 2019. [Online]. Available: https://www.vector.com/int/en/products-a-z/software/canoe/

[14] J.-S. Jhou, S.-H. Chen, W.-D. Tsay, and M.-C. Lai, "The implementation of OBD-II vehicle diagnosis system integrated with cloud computation technology," in *Proc. 2nd Int. Conf. Robot, Vis. Signal Process.*, Dec. 2013, pp. 9–12.

[15] B. Pattanaik and S. Chandrasekaran, "Recovery and reliability prediction in fault tolerant automotive embedded system," in *Proc. Int. Conf. Emerg. Trends Electr. Eng. Energy Manage. (ICETEEEM)*, Dec. 2012, pp. 257–262.

[16] S. Kelkar and R. Kamal, "Adaptive fault diagnosis algorithm for controller area network," *IEEE Trans. Ind. Electron.*, vol. 61, no. 10, pp. 5527–5537, Oct. 2014.

[17] H. Gawand, A. K. Bhattacharjee, and K. Roy, "Real time jitters and cyber physical system," in *Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI)*, Sep. 2014, pp. 2004–2008.

[18] A. Munir and F. Koushanfar, "Design and performance analysis of secure and dependable cybercars: A steer-by-wire case study," in *Proc. 13th IEEE Annu. Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2016, pp. 1066–1073.

[19] M. B. Nor Shah, A. R. Husain, H. Aysan, S. Punnekkat, R. Dobrin, and F. A. Bender, "Error handling algorithm and probabilistic analysis under fault for CAN-based Steer-by-Wire system," *IEEE Trans. Ind. Informat.*, vol. 12, no. 3, pp. 1017–1034, Jun. 2016.

[20] G. Marcon Zago and E. Pignaton de Freitas, "A quantitative performance study on CAN and CAN FD vehicular networks," *IEEE Trans. Ind. Electron.*, vol. 65, no. 5, pp. 4413–4422, May 2018.

[21] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2016, pp. 63–68.

[22] L. Zhang, Y. Lei, and Q. Chang, "Intermittent connection fault diagnosis for CAN using data link layer information," *IEEE Trans. Ind. Electron.*, vol. 64, no. 3, pp. 2286–2295, Mar. 2017.

[23] D. Heffernan and C. MacNamee, "Runtime observation of functional safety properties in an automotive control network," *J. Syst. Archit.*, vol. 68, pp. 38–50, Aug. 2016.

[24] A. Theissler, "Detecting known and unknown faults in automotive systems using ensemble-based anomaly detection," *Knowl.-Based Syst.*, vol. 123, pp. 163–173, May 2017.

[25] K. Becker, S. Voss, and B. Schätz, "Formal analysis of feature degradation in fault-tolerant automotive systems," *Sci. Comput. Program.*, vol. 154, pp. 89–133, Mar. 2018.
[26] W. Lu, Y. Li, Y. Cheng, D. Meng, B. Liang, and P. Zhou, "Early fault detection approach with deep architectures," *IEEE Trans. Instrum. Meas.*, vol. 67, no. 7, pp. 1679–1689, Jul. 2018.
[27] Z. Gao, C. Cecati, and S. X. Ding, "A survey of fault diagnosis and fault-tolerant techniques—Part I: Fault diagnosis with model-based and signal-based approaches," *IEEE Trans. Ind. Electron.*, vol. 62, no. 6, pp. 3757–3767, Jun. 2015.
[28] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*. Vienna, Austria: Springer, 2011.
[29] A. dos Santos Roque, C. Steinmetz, E. P. Freitas, and C. E. Pereira, "Modeling faults in communication protocols based on an aspect-oriented method," in *Proc. IEEE 15th Int. Conf. Ind. Informat. (INDIN)*, Jul. 2017, pp. 732–737.
[30] A. S. Roque, G. L. Nunes, E. P. Freitas, and C. E. Pereira, "Requirements specification and evaluation for transient faults in communication protocols based on RT-FRIDA framework," *IFAC-PapersOnLine*, vol. 51, no. 10, pp. 76–81, 2018.
[31] A. D. S. Roque, D. Pohren, E. P. Freitas, and C. E. Pereira, "An approach to address safety as non-functional requirements in distributed vehicular control systems," *J. Control, Autom. Electr. Syst.*, vol. 30, no. 5, pp. 700–715, Oct. 2019.
[32] E. P. Freitas, M. Wehrmeister, C. E. Pereira, F. R. Wagner, E. Silva, and F. Carvalho, "Using aspect-oriented concepts in the requirements analysis of distributed real-time embedded systems," in *Embedded System Design: Topics, Techniques and Trends*. Cham, Switzerland: Springer, 2007, pp. 221–230.
[33] T. J. Michelin, "Análise do impacto da comunicação via rede FlexRay em sistemas de controle," M.S. thesis, Elect. Eng., Federal University of Rio Grande do Sul, Porto Alegre, Brazil, 2014.
[34] O. Object Management Group. (2019). *OMG: The UML Profile for Modeling and Analysis of Real Time and Embedded System (MARTE)*. Accessed: 2019. [Online]. Available: http://www.omgmarte.org/
[35] C. Poussot-Vassal, "Robust LPV multivariable automotive global chassis control," Ph.D. dissertation, Automatic. Institut Nat. Polytechnique de Grenoble–INPG, Grenoble, France, 2008.
[36] J. M. G. da Silva, C. E. Pereira, and T. J. Michelin, "Performance analysis of distributed control systems using the FlexRay protocol," *IFAC Proc. Volumes*, vol. 47, no. 3, pp. 5252–5257, 2014.
[37] G. I. Mary, Z. C. Alex, and L. Jenkins, "Response time analysis of messages in controller area network: A review," *J. Comput. Netw. Commun.*, vol. 2013, pp. 1–11, Jan. 2013.
[38] M. Nahas, M. J. Pont, and M. Short, "Reducing message-length variations in resource-constrained embedded systems implemented using the controller area network (CAN) protocol," *J. Syst. Archit.*, vol. 55, nos. 5–6, pp. 344–354, May 2009.

Alexandre Dos Santos Roque (Member, IEEE) received the B.Sc. degree in computer science from Regional Integrated University—URI, Santo Angelo, Brazil, in 2005, the M.Sc. degree in production engineering with emphasis in industrial automation from the Federal University of Santa Maria—UFSM, Santa Maria, Brazil, in 2010, and the Ph.D. degree in electrical engineering from the Federal University of Rio Grande do Sul (UFRGS), Porto Alegre, Brazil. He is currently a member of the Research Group in Control, Automation, and Robotics—GCAR, UFRGS. He achieves part of the doctoral studies as a Researcher with the Institute of Industrial Automation and Software Engineering (IAS), University of Stuttgart, Germany, in 2019. His research interests include reliability, embedded systems, distributed real time control systems, and automotive communication protocols.

Nasser Jazdi (Member, IEEE) received the Diploma degree in electrical engineering in 1997, and the Ph.D. degree from the University of Stuttgart, Germany, in 2003. He joined the Institute of Industrial Automation and Software Engineering (IAS), University of Stuttgart, as the Deputy Head, a Researcher, and a Lecturer in 2003. His research interests include software reliability in the context of IoT, learning aptitude for industrial automation, and artificial intelligence in industrial automation. He is a member of the VDE Association for Electrical, Electronic & Information Technologies, the VDI-GPP Software Reliability Group, and the Berkeley Initiative in Soft Computing (BISC).

Edison Pignaton De Freitas (Member, IEEE) received the bachelor's degree in computer engineering from the Military Institute of Engineering, Brazil, in 2003, the M.Sc. degree in computer science from the Federal University of Rio Grande do Sul (UFRGS), Brazil, in 2007, and the Ph.D. degree in computer science and engineering from Halmstad University, Sweden, in 2011, in the area of wireless sensor networks. He is currently an Associate Professor with UFRGS, affiliated to the Graduate Programs in Electrical Engineering and Computer Science, acting as a member of the Research Group in Control, Automation and Robotics—GCAR, working in several research areas, such as industry automation, computer networks and real time systems.

Carlos Eduardo Pereira (Member, IEEE) received the B.S. degree in electrical engineering and the M.Sc. degree in computer science from Federal University of Rio Grande do Sul (UFRGS), Porto Alegre, and the Dr.-Ing. degree in electrical engineering from the University of Stuttgart, Germany, in 1995. He is currently a Full Professor with UFRGS. He is also the Director of Operations with EMBRAPII, Brazil. He has more than 400 technical publications on conferences and journals. He is a Council Member of IFAC. He received the Friedrich Wilhelm Bessel Research Award from the Alexander von Humboldt Foundation—Germany, in 2012. He is an Associate Editor of the Journal *Control Engineering Practice* and *Annual Reviews in Control* (Elsevier).